

Software Verification

Master Programme in Computer Science

Mário Pereira `mjp.pereira@fct.unl.pt`

Nova School of Science and Technology, Portugal

October 5, 2026

Lecture 4

based on previous editions by João Seco, Luís Caires, and Bernardo Toninho

based on lectures by Andrei Paskevich, Jean-Christophe Filliâtre and Claude Marché

The first handout is due for this Saturday, at 23:59.

Delivery method: submit your resolution to

<https://nsst-sv26.hotcrp.com>

You might need to create an account first.

Your submission is

- the ROCQ file with your solution (mandatory)
- a report explaining your decisions and difficulties (optional)

The **authors** field should provide information about the team members

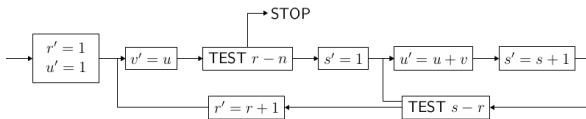
- email
- names with number
- affiliation

Verification of Imperative Programs

1. An historic perspective
2. The While language
3. Introduction to Hoare Logic
4. Introduction to the Why3 tool



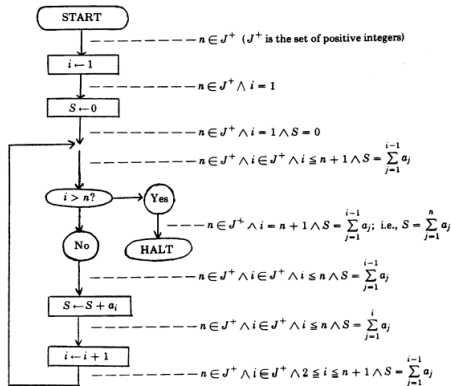
A. M. Turing. **Checking a large routine.** 1949.



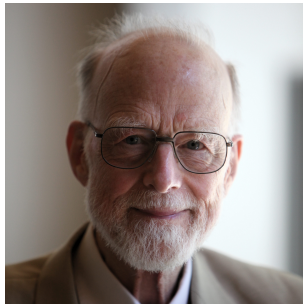
What does this program do?

"How can one check a routine in the sense of making sure that it is right? In order that the man who checks may not have too difficult a task the programmer should make a number of definite assertions which can be checked individually, and from which the correctness of the whole programme easily follows."

R. Floyd. **Assigning meaning to programs.** 1967.



"Robert Floyd's, "Assigning Meanings to Programs," opened the field of program verification. His basic idea was to attach so-called "tags" in the form of logical assertions to individual program statements or branches that would define the effects of the program based on a formal semantic definition of the programming language."



C. A. R. Hoare.

An axiomatic basis for computer programming.

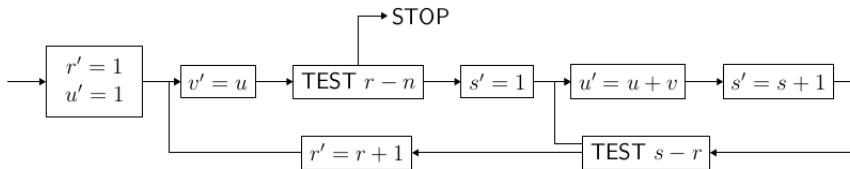
Commun. ACM, 1969

Proof of a program: FIND.

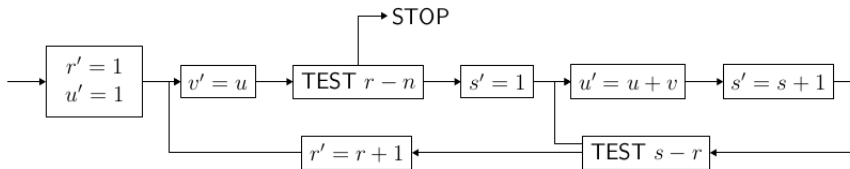
Commun. ACM, 1971

“Computer Programming is an exact science in that all the properties of a program and all consequences of executing it in any given environment can, in principle, be found out from the text of the program itself by means of purely deductive reasoning.”

Lets get back to Turing

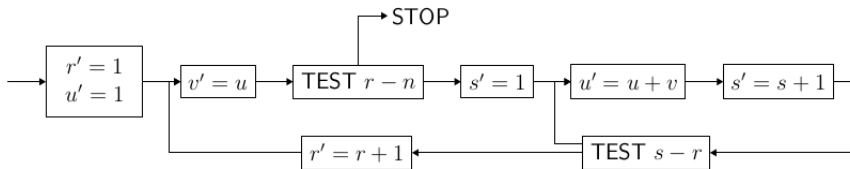


Lets get back to Turing



```
u := 1
for r = 0 to n - 1 do
  v := u
  for s = 1 to r do
    u := u + v
```

Lets get back to Turing



precondition { $n \leq 0$ }

$u := 1$

for $r = 0$ to $n - 1$ do

$v := u$

 for $s = 1$ to r do

$u := u + v$

postcondition { $u = \text{fact}(n)$ }

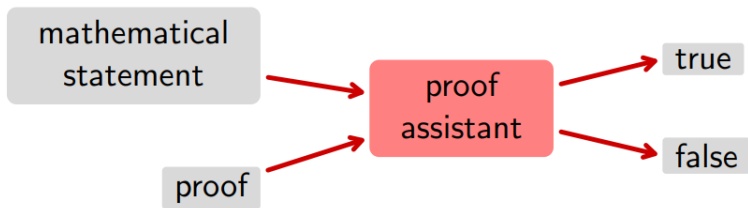
What do we do with this **implementation** and **logical specification**?

Deductive program verification is the art of turning the correctness of a program into a mathematical statement and then proving it.

We could perform **a manual proof** (as Turing and Hoare did) but it is long, tedious, and error-prone.

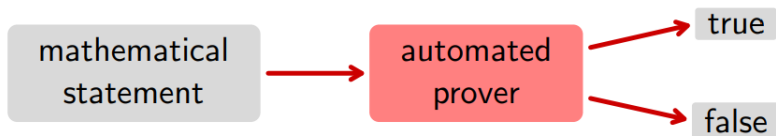
So we turn to tools that **mechanize mathematical reasoning**.

Build a proof and ask a **proof assistant** to check it.



Example: **Coq**, Isabelle, PVS, HOL Light

The dream:



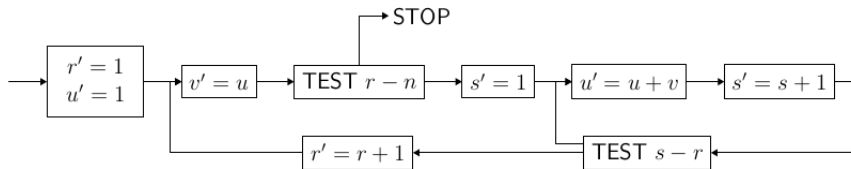
It is not possible to implement such a program
(Turing/Church, 1936, from Gödel)

Theorem: mathematicians will always have jobs!



Kurt Gödel

Lets get back to Turing, once again



precondition { $n \leq 0$ }

$u := 1$

for $r = 0$ to $n - 1$ do

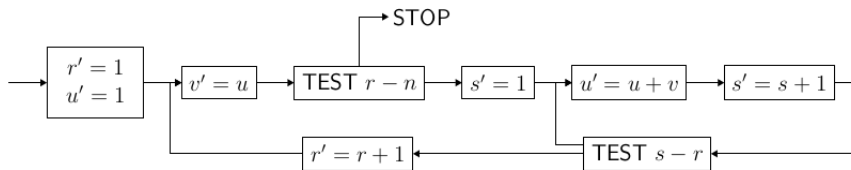
$v := u$

 for $s = 1$ to r do

$u := u + v$

postcondition { $u = \text{fact}(n)$ }

Lets get back to Turing, once again



precondition { $n \leq 0$ }

$u := 1$

for $r = 0$ to $n - 1$ do invariant { $u = \text{fact}(r)$ }

$v := u$

 for $s = 1$ to r do invariant { $u = s \times \text{fact}(r)$ }

$u := u + v$

postcondition { $u = \text{fact}(n)$ }

```
function fact(int) : int
```

```
axiom fact0: fact(0) = 1
```

```
axiom factn:  $\forall n:\text{int}. n \geq 1 \rightarrow \text{fact}(n) = n * \text{fact}(n-1)$ 
```

```
goal vc:  $\forall n:\text{int}. n \geq 0 \rightarrow$ 
```

```
  (0 > n - 1  $\rightarrow$  1 = fact(n))  $\wedge$ 
```

```
  (0  $\leq$  n - 1  $\rightarrow$ 
```

```
    1 = fact(0)  $\wedge$ 
```

```
    ( $\forall u:\text{int}.$ 
```

```
      ( $\forall r:\text{int}. 0 \leq r \wedge r \leq n - 1 \rightarrow u = \text{fact}(r) \rightarrow$ 
```

```
        (1 > r  $\rightarrow u = \text{fact}(r + 1)) \wedge$ 
```

```
        (1  $\leq$  r  $\rightarrow$ 
```

```
          u = 1 * fact(r)  $\wedge$ 
```

```
          ( $\forall u1:\text{int}.$ 
```

```
            ( $\forall s:\text{int}. 1 \leq s \wedge s \leq r \rightarrow u1 = s * \text{fact}(r) \rightarrow$ 
```

```
              ( $\forall u2:\text{int}.$ 
```

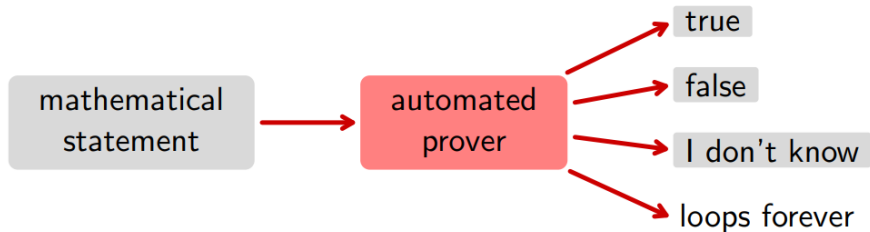
```
                u2 = u1 + u  $\rightarrow u2 = (s + 1) * \text{fact}(r))) \wedge$ 
```

```
                (u1 = (r + 1) * fact(r)  $\rightarrow u1 = \text{fact}(r + 1)))))) \wedge$ 
```

```
  (u = fact((n - 1) + 1)  $\rightarrow u = \text{fact}(n)))$ 
```

```
function fact(int) : int  
axiom fact0: fact(0) = 1
```

```
goal vc:  $\forall n:\text{int}. n \geq 0 \rightarrow$   
   $(0 > n - 1 \rightarrow 1 = \text{fact}(n)) \wedge$ 
```



Examples: Z3, CVC5, Alt-Ergo, Vampire, SPASS, etc.

Contract-based Verification

A prime on Hoare Logic

```
var sum := 1;
var count := 0;
while sum <= n {
    count := count + 1;
    sum := sum + 2 * count + 1;
}
return count;
```

What is the result of this expression for a given n ?

```
var sum := 1;
var count := 0;
while sum <= n {
    count := count + 1;
    sum := sum + 2 * count + 1;
}
return count;
```

What is the result of this expression for a given n ?

Informal specification:

- at the end, `count` contains the truncated square root of n
- for instance, given $n = 42$, the returned value is 6

A statement about program correctness:

$$\{P\} e \{Q\}$$

P precondition property

e expression

Q postcondition property

What is the meaning of a Hoare triple?

$\{P\} e \{Q\}$ if we execute e in a state that satisfies P ,
then the computation either diverges
or terminates in a state that satisfies Q

This is **partial correctness**: we say nothing about termination.

Examples of valid Hoare triples for partial correctness:

- $\{x = 1\} \ x := x + 2 \ \{x = 3\}$
- $\{x = y\} \ x + y \ \{\text{result} = 2y\}$
- $\{\exists v. x = 4v\} \ x + 42 \ \{\exists w. \text{result} = 2w\}$
- $\{\text{true}\} \ \text{while true do skip} \ \{\boxed{\boxed{\text{false}}}\}$

Examples of valid Hoare triples for partial correctness:

- $\{x = 1\} \ x := x + 2 \ \{x = 3\}$
- $\{x = y\} \ x + y \ \{\text{result} = 2y\}$
- $\{\exists v. x = 4v\} \ x + 42 \ \{\exists w. \text{result} = 2w\}$
- $\{\text{true}\} \ \text{while true do skip} \ \{\boxed{\boxed{\text{false}}}\}$
 - after this loop, *everything* is trivially verified
 - ergo: not proving termination can be fatal

Examples of valid Hoare triples for partial correctness:

- $\{x = 1\} \ x := x + 2 \ \{x = 3\}$
- $\{x = y\} \ x + y \ \{\text{result} = 2y\}$
- $\{\exists v. x = 4v\} \ x + 42 \ \{\exists w. \text{result} = 2w\}$
- $\{\text{true}\} \ \text{while true do skip} \ \{\boxed{\boxed{\text{false}}}\}$
 - after this loop, *everything* is trivially verified
 - ergo: not proving termination can be fatal

In our square root example:

$$\{?\} \text{ISQRT} \{?\}$$

Examples of valid Hoare triples for partial correctness:

- $\{x = 1\} \ x := x + 2 \ \{x = 3\}$
- $\{x = y\} \ x + y \ \{\text{result} = 2y\}$
- $\{\exists v. x = 4v\} \ x + 42 \ \{\exists w. \text{result} = 2w\}$
- $\{\text{true}\} \ \text{while true do skip} \ \{\boxed{\boxed{\text{false}}}\}$
 - after this loop, *everything* is trivially verified
 - ergo: not proving termination can be fatal

In our square root example:

$$\{n \geq 0\} \text{ISQRT} \ \{?\}$$

Examples of valid Hoare triples for partial correctness:

- $\{x = 1\} \ x := x + 2 \ \{x = 3\}$
- $\{x = y\} \ x + y \ \{\text{result} = 2y\}$
- $\{\exists v. x = 4v\} \ x + 42 \ \{\exists w. \text{result} = 2w\}$
- $\{\text{true}\} \ \text{while true do skip} \ \{\boxed{\boxed{\text{false}}}\}$
 - after this loop, *everything* is trivially verified
 - ergo: not proving termination can be fatal

In our square root example:

$$\{n \geq 0\} \text{ ISQRT } \{\text{result}^2 \leq n < (\text{result} + 1)^2\}$$

While – A Simple Imperative Programming Language

$E ::=$	Expressions
num	Integer
$ \quad x$	Variable
$ \quad E + E \mid \dots$	Integer operations
$ \quad E < E \mid \dots$	Relational operations
$ \quad E \text{ and } E \mid \dots$	Boolean operations
$P ::=$	Programs
$skip$	No operation
$ \quad x := E$	Assignment
$ \quad P; P$	Sequential Composition
$ \quad \text{if } E \text{ then } P \text{ else } P$	Conditional
$ \quad \text{while } E \text{ do } P$	Iteration

An **imperative** program is a **state transformer**.
It transforms an **initial** state into a **target** state.

What is a program state? A **mapping** of state variables to values:

$$\sigma = \{x \mapsto 1; y \mapsto 2; w \mapsto 3\}$$

An imperative program transforms states into states

$$P \triangleq x := y + x; w := w - x$$

If P is executed in state σ it yields state σ' where

$$\sigma' = \{x \mapsto 3; y \mapsto 2; w \mapsto 0\}$$

We say that P transforms σ into σ' .

Initially (1970): **axiomatic semantics** of programs

Inference rules to construct valid triples:

$$\overline{\{P\} \text{ skip } \{P\}}$$

$$\overline{\{P[x \mapsto t]\} x := t \{P\}}$$

$$\frac{\{P\} e_1 \{Q\} \quad \{Q\} e_2 \{R\}}{\{P\} e_1 ; e_2 \{R\}}$$

Notation $P[x \mapsto t]$: replace in P every occurrence of x with t

Consequence rule:

$$\frac{\models P \Rightarrow P' \quad \{P'\} e \{Q'\} \quad \models Q' \Rightarrow Q}{\{P\} e \{Q\}}$$

Example: proof of $\{x = 1\} x := x + 2 \{x = 3\}$

$$\frac{\models x = 1 \rightarrow x + 2 = 3 \quad \frac{(x = 3)[x \mapsto x + 2] \equiv x + 2 = 3}{\vdots} \quad \{x + 2 = 3\} x := x + 2 \{x = 3\}}{\{x = 1\} x := x + 2 \{x = 3\}}$$

Rules for **if** and **while**:

$$\frac{\{P \wedge t\} e_1 \{Q\} \quad \{P \wedge \neg t\} e_2 \{Q\}}{\{P\} \text{if } t \text{ then } e_1 \text{ else } e_2 \{Q\}}$$

$$\frac{\{J \wedge t\} e \{J\}}{\{J\} \text{while } t \text{ do } e \{J \wedge \neg t\}}$$

Formula J is a **loop invariant**. (more on this later)

Finding a right invariant is a **major difficulty**.

A program proof in Hoare logic **adds assertions between program statements**, making sure that all Hoare triples are satisfied/valid.

Consider the following code snippet:

```
if (x > y) then
  z := x
else
  z := y
```

A Hoare Logic “proof” may look like

```
{ true }  
if (x > y) then  
  { (x > y) }  
  z := x  
  { (x > y) ∧ (z == x) }  
else  
  { (x ≤ y) }  
  z := y  
  { (x ≤ y) ∧ (z == y) }  
{ (x > y) ∧ (z == x) || (x ≤ y) ∧ (z == y)}  
{ z == max(x, y) }
```

Homework: use Hoare rules to check this derivation is valid.

Rules of Hoare Logic (general form)

The inference rules of Hoare logic are used to derive (valid) Hoare triples given some already derived Hoare triples.

$$\frac{\{A_1\} P_1 \{B_1\} \quad \{A_n\} P_n \{B_n\}}{\{A\} C(P_1, \dots, P_n) \{B\}}$$

What is nice here:

- the program in the conclusion contains the **subprograms** $P_1, \dots, P_n$ as components
- we derive properties of the **composite** from the properties of its parts (compositionality)
- pretty much the same as with a type system

We may want to delegate some functionality to a **function**:

let $f (v_1 : \tau_1) \dots (v_n : \tau_n) : \tau \ C = e$ defined function

val $f (v_1 : \tau_1) \dots (v_n : \tau_n) : \tau \ C$ abstract function

Function behavior is specified with a **contract**:

$C ::=$	requires P	precondition
	writes $x_1 \dots x_n$	modified global references
	ensures Q	postcondition

Declarations annotated with **pre**- and **post**- conditions.

A program P is a **set** of **function declarations**.

Each **function declaration** is validated as follows:

1. **assume** its pre-condition
2. **prove** its post-condition.

$$\frac{\{P \ v_1 \ \dots \ v_n\} e \{Q \ v_1 \ \dots \ v_n \ \mathcal{R}\}}{\text{let } f \ (v_1 : \tau_1) \ \dots \ (v_n : \tau_n) : \tau \\ \text{requires } P \ v_1 \ \dots \ v_n \\ \text{ensures } Q \ v_1 \ \dots \ v_n \ \mathcal{R} = e}$$

$\text{let } f (v_1 : \tau_1) \dots (v_n : \tau_n) : \tau$
 $\text{requires } P \ v_1 \dots v_n$
 $\text{ensures } Q \ v_1 \dots v_n \ \mathcal{R} = e$

$$\frac{A \Rightarrow P \ e_1 \dots e_n \quad Q \ e_1 \dots e_n \ \mathcal{R} \Rightarrow B[x \mapsto r]}{\{A\} x := f \ e_1 \dots e_n \{B\}}$$

Each **function call** is validated as follows:

1. **prove** the (instantiated) pre-condition of f

$$A \Longrightarrow P \ e_1 \dots e_n$$

2. **assume** the (instantiated) post-condition of f

$$Q \ e_1 \dots e_n \ \mathcal{R} \Longrightarrow B[x \mapsto \mathcal{R}]$$

$\text{let } f(v_1 : \tau_1) \dots (v_n : \tau_n) : \tau$
 $\text{requires } P \ v_1 \dots v_n$
 $\text{ensures } Q \ v_1 \dots v_n \ \mathcal{R} = e$

$$\frac{A \Rightarrow P \ e_1 \dots e_n \quad Q \ e_1 \dots e_n \ \mathcal{R} \Rightarrow B[x \mapsto r]}{\{A\} x := f \ e_1 \dots e_n \{B\}}$$

Calls are **opaque**. We only know what is in the post-condition.

Verification with function calls is **modular**.

Derive

1.

$$\{y > 0\} x := y \{x > 0 \wedge y == x\}$$

2.

$$\{x == y\} x := 2 * x \{2 * y == x\}$$

3.

$$\{P(y) \wedge Q(w)\} x := y; y := w; w := x \{P(w) \wedge Q(y)\}$$

Introduction to the Why3 tool

demo

- R. W. Floyd.
[Assigning Meanings to Programs.](#)
Proc. of Symposia in Applied Mathematics, 1967
- C. A. R. (Tony) Hoare.
[An Axiomatic Basis for Computer Programming.](#)
Communications of the ACM, 1969
- E. W. Dijkstra.
[A Discipline of Programming.](#) Prentice Hall, 1976
- J. B. Almeida, M. J. Frade, J. S. Pinto, S. Melo de Sousa.
[Rigorous Software Development.](#) Springer, 2011
- The Why3 Development Team.
[The Why3 Platform.](#) Version 1.8, September 2025